

CHIP-8 Emulator in C++

A 1-Week Portfolio Project Guide

Scope: build a working CHIP-8 interpreter in modern C++ that runs real game ROMs (Pong, Tetris, Space Invaders) with SDL2 for graphics and input.

Designed for someone whose C++ background is data structures & algorithms - this guide tells you what to build each day, which parts to write yourself, what to delegate to AI tools, and the talking points that make this project interview-worthy.

1. Why This Project

Emulators are the 'hello world' of low-level programming credibility. A CHIP-8 emulator shows you understand how a CPU actually works - fetch, decode, execute - plus memory, bitwise operations, timers, and hardware input/output. It is the standard recommendation on r/EmuDev as the first emulation project before Game Boy or NES territory.

It is also the rare project with an unbeatable demo: a screen recording of a real game running on code you wrote. Recruiters glance at GIFs; interviewers ask about your decode loop.

What you will be able to say in interviews

- How a CPU executes instructions: fetch -> decode -> execute, program counter, call stack.
- Why you chose a switch-based dispatch vs. a function-pointer table (and the trade-offs).
- How sprites are drawn via XOR and how collision detection falls out of it for free.
- How you decoupled CPU speed (~500-700 instr/sec) from the 60 Hz display/timer refresh.
- How you debugged opcode-level bugs using known test ROMs.

2. CHIP-8 Architecture Cheatsheet

CHIP-8 is not a real CPU - it is a 1970s virtual machine spec, which is why it is perfect for a first emulator. The whole machine fits in one class:

```
class Chip8 {
    uint8_t memory[4096]; // RAM; programs load at 0x200
    uint8_t V[16];        // registers V0-VF (VF = flag/carry)
    uint16_t I;            // index register (addresses)
    uint16_t pc;           // program counter, starts at 0x200
    uint16_t stack[16];    // call stack
    uint8_t sp;            // stack pointer
    uint8_t delayTimer, soundTimer; // decrement at 60 Hz
    uint32_t display[64*32]; // monochrome framebuffer
    uint8_t keypad[16];    // hex keypad 0-F
};
```

Memory	4 KB, addresses 0x000-0xFFFF. Font sprites conventionally stored at 0x050. ROM loads at 0x200.
---------------	--

Registers	V0-VE general purpose (8-bit). VF doubles as carry/flag - some ops overwrite it.
Instructions	Always 2 bytes, big-endian. Read opcode = memory[pc] << 8 memory[pc+1].
Display	64x32 pixels, monochrome. DXYN draws an 8xN sprite at (Vx, Vy) using XOR; VF = pixels turned off (collision).
Timers	Delay + sound timers tick down at 60 Hz, independent of CPU speed. Sound > 0 = beep.
Keypad	16 keys 0x0-0xF. Original layout maps awkwardly to keyboards; use 1234/QWER/ASDF/ZXCV.
Op decode	Split the 16-bit opcode into nibbles: n1 = op>>12, x = (op>>8)&0xF, y = (op>>4)&0xF, n = op&0xF, nn = op&0xFF, nnn = op&0xFFF.

3. Setup (Do This Before Day 1)

On Windows: MSVC (from Visual Studio) + CMake + SDL2. Get the toolchain working and render an empty window before you write any emulation code.

- Any C++17 toolchain works: Visual Studio Community (MSVC), or MinGW g++.
- SDL2: download SDL2-devel-x.y.z-VC.zip from [github.com/libsdl-org/SDL](https://github.com/libsdl-org/SDL/releases) releases and extract it into the repo (e.g. external/SDL2-x.y.z/) - prebuilt MSVC libs, no package manager needed.
- CMakeLists.txt: point SDL2_DIR at the extracted cmake/ folder, then find_package(SDL2 CONFIG REQUIRED) and link SDL2::SDL2.
- VS Code route: install the CMake Tools extension, pick the MSVC amd64 kit, and set cmake.debugConfig cwd to \${workspaceFolder} in .vscode/settings.json so relative ROM paths resolve.
- Skeleton repo: src/main.cpp, src/chip8.cpp, include/chip8.h, roms/ folder, CMakeLists.txt.
- Milestone: an SDL2 window opens, stays open, and closes cleanly on the X button.

AI is fine for all of this. CMake toolchain files and SDL window boilerplate are pure plumbing - delegate freely. The learning starts at the emulator core.

4. The 7-Day Plan

Day 1 - Skeleton, ROM loading, and the main loop

Goal: Chip8 class with all state, loadRom() reading bytes into memory[0x200], and the emulation loop running at a fixed instruction rate.

- Load the fontset (16 five-byte sprites, 0-F) into memory at 0x050. Find the standard fontset bytes in the references - copy them, they are data, not logic.
- loadRom(): open file binary, read into memory starting at 0x200, set pc = 0x200.
- Write cycle(): fetch opcode, decode, execute, pc += 2. Leave execute() as a stub switch.
- Wire a main loop that calls cycle() N times per frame and ticks timers at 60 Hz.

Learning value:

How a binary file becomes machine state. Why pc starts at 0x200. The classic mistake you will hit: treating the two-byte opcode as little-endian and decoding garbage. This is also where you internalize that a CPU is just a loop over memory.

You write vs. AI writes:

You: memory layout, loadRom, fetch, the loop structure. AI: SDL window init, CMake plumbing, fontset byte table.

Day 2 - First opcodes and the draw instruction

Goal: Enough opcodes to run the IBM logo test ROM - the 'hello world' screenshot of CHIP-8 dev.

- Implement: 00E0 (clear), 1NNN (jump), 6XNN (Vx = NN), 7XNN (Vx += NN), ANNN (I = NNN), DXYN

(draw).

- DXYN is the heart: for each of N rows, read a sprite byte from memory[I+row]; each set bit XORs a pixel; if a pixel flips off, set VF = 1. Wrap coordinates at screen edges.
- Render display[] to an SDL texture scaled up ~10x.
- Run the IBM logo ROM. Seeing the logo appear is the moment this stops being abstract.

Learning value:

Bitwise ops in anger: masking nibbles, XOR drawing, why VF-as-collision works. You will also learn sprite layout (each row = 8 pixels = 1 byte) - the same trick real hardware uses.

You write vs. AI writes:

You: every opcode, especially DXYN - debug it yourself even when it is painful; that debugging IS the skill. AI: SDL texture/render boilerplate, explaining 'why is my sprite shifted by one' when you are stuck.

Day 3 - Control flow: skips, calls, and the ALU group

Goal: Full branching and arithmetic so structured programs (loops, conditionals, subroutines) run.

- Skips: 3XNN, 4XNN, 5XY0, 9XY0 - all 'skip next instruction if condition' (pc += 4 instead of 2).
- Calls: 2NNN pushes pc to stack then jumps; 00EE pops and jumps back. Do not forget sp++ / sp--.
- ALU group 8XY0-8XYE: assign, OR, AND, XOR, add w/ carry, sub w/ borrow, shifts. VF carries the flag.
- Read the quirk note: on classic CHIP-8, 8XY6/8XYE shift Vx but use Vy's original value. Pick one behavior and document it.

Learning value:

How call stacks actually work (you have written stacks for DSA; now you use one to keep a machine alive). Carry/borrow flags - what 'unsigned underflow' really means. Your first spec ambiguity decision - real engineering is full of them.

You write vs. AI writes:

You: all of it - these are 1-3 lines each and high learning density. AI: sanity-checking your flag semantics against Cowgod's reference when behavior looks wrong.

Day 4 - Timers, random, and the FX** grab-bag

Goal: Finish the opcode table except input; run the full opcode test ROM.

- BNNN (jump NNN + V0), CXNN (Vx = rand() & NN) - use <random>, not rand().
- FX07/FX15/FX18: read/set delay timer, set sound timer.
- FX1E (I += Vx - watch for the undocumented VF-overflow quirk), FX29 (I = font address of Vx).
- FX33: BCD - store hundreds/tens/ones of Vx at memory[I..I+2]. Trickiest pure-logic opcode.
- FX55/FX65: dump/load V0..Vx to/from memory at I. Classic quirk: whether I increments.
- Run a flag/opcode test ROM (chip8-test-suite) and diff expected vs. actual.

Learning value:

BCD conversion is a tiny algorithm embedded in a machine spec - nice DSA crossover. Timer decoupling teaches you why games run at a fixed tick rate. Test ROMs teach regression-driven debugging: the test tells you which opcode family is wrong before you look at a single game.

You write vs. AI writes:

You: FX33 and the flag semantics - these generate the best interview stories. AI: mapping test-ROM failures to the responsible opcode group (it is good at 'symptom -> likely cause').

Day 5 - Input and real games

Goal: Playable Pong. This is demo day.

- EX9E / EXA1: skip if key Vx is (not) pressed.
- FX0A: blocking wait-for-key - halt cycle() until a keypress, store it in Vx. Simplest correct approach: set a 'waiting' flag and resume on keydown.
- Map keyboard 1234/QWER/ASDF/ZXCV to the hex keypad; update keypad[] on keydown/keyup.
- Tune instruction rate (~500-700 Hz) so games are playable - too fast and Pong is unplayable.

- Run Pong, Tetris, and a breakout ROM. Fix whatever breaks - it will be a quirk or input edge case.

Learning value:

Blocking input forces you to think about the machine's execution model - you cannot just `sleep()` inside `cycle()`. Rate-limiting teaches instruction throughput vs. wall-clock time. Real ROMs will expose every shortcut you took earlier; fixing them is where the depth comes from.

You write vs. AI writes:

You: the input model and FX0A semantics. AI: SDL event-loop wiring, and 'why does Pong flicker' brainstorming - but verify theories in the code yourself.

Day 6 - Polish and the demo artifact

Goal: A repo that looks like an engineer owns it, plus a recorded demo.

- README: architecture diagram (memory map + cycle flow), controls table, build instructions, opcode coverage table, quirks you chose and why.
- Record a GIF/video: IBM logo -> opcode test pass -> 10s of Pong gameplay. ScreenToGif or OBS.
- Clean the code: remove dead code, consistent naming, comments only where the spec is subtle.
- Optional flags: `--rom path`, `--rate`, `--debug` to print decoded opcodes.

Learning value:

The README is what gets read - write the design-decisions section yourself, it doubles as interview prep. A `--debug` trace flag also gives you a second demo: the machine's internals.

You write vs. AI writes:

AI: good at README structure suggestions and proofreading - but the 'decisions and trade-offs' section must be your words, because interviewers will probe exactly those sentences.

Day 7 - Buffer, stretch, and talking points

Goal: Margin for the bug that ate a day, plus one stretch feature.

- Stretch pick 1: a step/debug mode - pause, single-step, dump registers. Cheap to build, great to demo.
- Stretch pick 2: sound (a square-wave beep when `soundTimer > 0` via SDL audio).
- Stretch pick 3: SUPER-CHIP hi-res mode - only if everything else is done.
- Write down your 5 talking points (section 6) while the details are fresh.

Learning value:

A debugger turns your project from 'runs games' to 'tooling for understanding a machine' - a noticeable tier jump on a resume.

You write vs. AI writes:

AI: fine for the stretch features - you have earned the delegation budget by now.

5. Opcode Reference - Suggested Implementation Order

Batch 1 (D2)	00E0, 1NNN, 6XNN, 7XNN, ANNN, DXYN -> IBM logo
Batch 2 (D3)	3XNN, 4XNN, 5XY0, 9XY0, 2NNN, 00EE, 8XY0-8XYE
Batch 3 (D4)	BNNN, CXNN, FX07, FX15, FX18, FX1E, FX29, FX33, FX55, FX65
Batch 4 (D5)	EX9E, EXA1, FX0A
Skip	0NNN (RCA machine call - every modern ROM ignores it)

The quirks that WILL bite you

- 8XY6 / 8XYE (shifts): original COSMAC VIP used Vy's value then shifted into Vx; later interpreters shift Vx in place. Modern ROMs often expect the in-place version. Make it a config flag if you want to be fancy.
- FX55 / FX65 (memory dump/load): VIP incremented I by x+1; most emulators leave I unchanged. Modern

ROMs expect I unchanged.

- DXYN screen wrap: sprites wrap at edges. Off-by-one here = distorted sprites.
- Big-endian opcodes: opcode = (mem[pc] << 8) | mem[pc+1]. Reversed = nonsense decode.

6. Making It Count on a Resume

Resume bullet (adapt to taste)

- Built a CHIP-8 emulator/interpreter in C++17 running real game ROMs (Pong, Tetris) - implemented full opcode set, 4KB memory model, call stack, 60Hz timers, and XOR-sprite rendering with SDL2.
- Validated correctness against the chip8-test-suite ROMs; added a step-mode debugger for instruction tracing.

The questions interviewers actually ask - prep answers

- 'Walk me through your execute loop.' -> fetch/decode/execute, nibble extraction, switch dispatch.
- 'Why a switch instead of std::function table / jump table?' -> readability, compiler already emits jump tables, easier debugging.
- 'How does collision detection work?' -> XOR draw: VF=1 if any set pixel got turned off. Elegant spec detail - show you noticed it.
- 'How did you handle timers?' -> decoupled 60 Hz tick from instruction rate; sound timer edge cases.
- 'What was the hardest bug?' -> have ONE specific story: a flag quirk, an off-by-one in DXYN, or endianness - with how you isolated it (test ROM, debug trace).
- 'What would you do next?' -> SUPER-CHIP, save states, a disassembler view, or start a Game Boy emulator.

7. AI-Usage Strategy (So You Actually Learn It)

The risk with AI-assisted projects is a demo you cannot defend in an interview. The split that keeps this honest:

Delegate to AI

- Build plumbing: CMakeLists.txt, vcpkg manifest, compiler flags.
- SDL2 boilerplate: window/renderer/texture setup, event pump skeleton.
- Fontset byte table (it is data), README proofreading, error-message archaeology.
- Explaining concepts: 'explain fetch-decode-execute', 'why does XOR drawing detect collisions'.

Write yourself (non-negotiable)

- Every opcode implementation, the decode switch, DXYN, the call stack, FX0A input model.
- The bug fixes. When a ROM misbehaves, form a hypothesis, then use AI to check it - not to generate the fix wholesale.
- The design-decisions section of the README.

Two rules that preserve learning

- After AI writes anything, ask: 'what breaks if I change X?' and 'why Y over Z?' If you cannot answer, do not commit it yet.
- If you could not whiteboard the opcode loop from memory, the project is not done - regardless of whether the games run.

8. Full Opcode Quick Reference

Every standard CHIP-8 instruction. Read these against docs_cowgod_chip8.txt (the full spec, saved locally) while implementing - the one-liners here are summaries, the spec is the contract.

System / flow

```
00E0  clear display
00EE  return: pc = stack[--sp]
0NNN  SYS addr - ignore (legacy)
1NNN  jump: pc = nnn
2NNN  call: stack[sp++] = pc, pc = nnn
BNNN  jump to nnn + V[0]
```

Skips (skip = pc += 4 instead of +2)

```
3XNN  skip if V[x] == nn
4XNN  skip if V[x] != nn
5XY0  skip if V[x] == V[y]
9XY0  skip if V[x] != V[y]
```

Registers / ALU (8XY? - last nibble picks the op)

```
6XNN  V[x] = nn
7XNN  V[x] += nn (no carry flag)
8XY0  V[x] = V[y]
8XY1  V[x] |= V[y]
8XY2  V[x] &= V[y]
8XY3  V[x] ^= V[y]
8XY4  V[x] += V[y]; VF = carry
8XY5  V[x] -= V[y]; VF = NOT borrow
8XY6  V[x] = V[y] >> 1; VF = shifted-out bit (quirk: some shift Vx)
8XY7  V[x] = V[y] - V[x]; VF = NOT borrow
8XYE  V[x] = V[y] << 1; VF = shifted-out bit (quirk: some shift Vx)
```

I / misc / draw

```
ANNN  I = nnn
CXNN  V[x] = rand() & nn
DXYN  draw n rows at (V[x], V[y]) from memory[I]; VF = collision
```

Keys

```
EX9E  skip if keypad[V[x]] pressed
EXA1  skip if keypad[V[x]] not pressed
FX0A  block until keypress; store key in V[x]
```

Timers / memory (FX?? - switch on nn)

```
FX07  V[x] = delayTimer
FX15  delayTimer = V[x]
FX18  soundTimer = V[x]
FX1E  I += V[x]
FX29  I = FONTSET_START + V[x] * 5
FX33  memory[I..I+2] = BCD digits of V[x]
FX55  memory[I..I+x] = V[0..x] (quirk: VIP did I += x+1)
FX65  V[0..x] = memory[I..I+x] (quirk: same I question)
```

9. Resources

- Cowgod's CHIP-8 Technical Reference - the canonical opcode spec. Saved locally as docs_cowgod_chip8.txt in this repo (original: devernay.free.fr/hacks/chip8/C8TECH10.HTM). Work Days 3-4 straight from section 3.1.
- Tobias V. Langhoff's 'Guide to making a CHIP-8 emulator' (tobiasvl.github.io) - the best modern walkthrough, covers quirks honestly.
- Austin Morlan's C++ tutorial - useful for structure if you get lost.
- chip8-test-suite (github.com/Timendus/chip8-test-suite) - opcode/flag/quirk test ROMs. Your regression suite.
- ROMs: github.com/kripod/chip8-roms - Pong, Tetris, Space Invaders, IBM logo.
- SDL2 docs for texture streaming - you only need ~3 API calls all project.